Data Structures & Algorithms
Assignment # 1 (CLO-2.1)

Objective

Students will explore:

- To understand the fundamental concepts of Data Structures, ADTs, and OOP as tools for organizing and managing data.
- To develop the ability to explain how arrays, dynamic containers, and List ADT are used in real-world applications.
- To practice designing class hierarchies and mapping data structures to problem solutions.
- To build a foundation for implementing C++ programs later by first learning how to explain and design solutions.
- To encourage clarity of thought, logical reasoning, and structured problem-solving.

Instructions: Make sure to follow these instructions carefully to avoid losing marks.

Topic: Your assignment is based on the topic:

"Introduction to Data Structures, ADTs, Classes, Arrays, and List ADT"

Format

- Students must write the answers **by hand** on paper (using pen/marker).
- They can then scan or take clear photos of each page and combine into a **single PDF file** (using apps like CamScanner, Adobe Scan, or phone camera with PDF converter).
- Finally, they must upload the handwritten PDF on the portal (Moellim in your case).

Content Guidelines

1. Define Data Structure, ADT, and List ADT.
2. Explain how OOP concepts (classes, information hiding, constructors) apply to a hospital management system.
3. Draw a class hierarchy diagram showing Person, Patient, Doctor, and Container ADT.
4. Write pseudocode for adding and searching patients using List ADT operations.
5. Ensure clarity, logical sequence, and proper structure in explanations and diagrams.
6. Label all diagrams neatly and explain them briefly in the text.

Assignment Structure & Guidelines

Q1. Define **Data Structure** and **Abstract Data Type (ADT)**. Explain their importance in problem-solving with one simple example from a hospital management system.

Q2. Define **List ADT**. Explain its common operations (insert, remove, retrieve, traverse, search) and how they help manage patient records.

Q3. Design a **Hospital Management System** using OOP concepts:

- Create a base class Person with private data members (name, id, age).
- Derive Patient and Doctor classes with their own private members.
- Use **constructors** to initialize objects.
- Use **information hiding** (private members, public methods).
- Draw a **class hierarchy diagram** and explain it briefly.

Q4. Explain how a **Dynamic Container Class** works using **pointers** and **dynamic memory** (new/delete). Why is it better than a static array for storing an unknown number of patients?

Q5. Write **pseudocode** for the following List ADT operations in the hospital system:

- Add a new patient
- Search for a patient by ID
- Remove a patient by ID

Explain which operations you used and why.

Submission

- Submit the assignment in **PDF format and .cpp file** via the Moellim portal by **23-09-2026**.
- Ensure the file name follows the format: **SAP_ID_Data Structures_Assignment #1**
- **Plagiarism:** This is an individual assignment. Ensure the work is original and properly referenced if external sources are used. Assignments with plagiarism will not be accepted and will result in a zero grade.

- **Deadline:** Late submissions will incur a **5% deduction per day**.

Output:

```
===== Hospital Management System =====
1. Add Patient
2. Remove Patient
3. Search Patient
4. Display All Patients
5. Exit
Enter choice: 4

===== Display All Patients =====
Name: ali | ID: 987 | Age: 45 | Disease: fiver | Room: 23
```

Rubrics (Total = 10 Marks)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (2)	Poor (1)
Concept Understanding	Clear, deep explanation with examples.	Good explanation, minor gaps.	Basic understanding.	Limited explanation.	No understanding shown.
ADTs & List ADT	Complete, correct, and well-structured explanation of all operations.	Mostly correct, some errors.	Basic attempt, partially correct.	Incomplete/unclear.	Not attempted.
Diagrams (Class Hierarchy)	Accurate, neat, and well-labeled.	Neat diagrams, minor mistakes.	Basic diagrams, labels missing.	Incomplete diagrams.	No diagrams given.
Pseudocode & Logic	Complete, correct, and well-structured pseudocode.	Mostly correct, some errors.	Basic attempt, partially correct.	Incomplete/unclear.	Not attempted.
Presentation	Well-structured, error-free.	Clear but some mistakes.	Acceptable but weak flow.	Poor structure, many mistakes.	Unreadable.

Marks Distribution

Criteria	Marks
Concept Understanding (Q1, Q2)	2
ADTs & List ADT (Q2, Q5)	2

Criteria	Marks
Diagrams (Class Hierarchy) (Q3)	2
Pseudocode & Logic (Q5)	2
Dynamic Container & Pointers (Q4)	1
Presentation	1
Total	**10**